Common setup (run once)

```
pip install nltk
```

```
import nltk

# Download the NLTK datasets/models needed by the lab programs.
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('brown')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('wordnet')
nltk.download('omw-1.4')
nltk.download('universal_tagset')
nltk.download('conll2000')
nltk.download('maxent_ne_chunker_tab')
nltk.download('words')
```

Run each program in Jupyter Notebook, Google Colab, or VS Code with Python installed. Some programs download corpora on first run and need an internet connection.

3. Generate word forms from root and suffix information using morphological analysis and generation

Aim: To demonstrate morphological analysis and generation as inverse processes.

```
# Define root words and their possible suffixes.
morphology = {
    'play': ['s', 'ed', 'ing', 'er'],
    'walk': ['s', 'ed', 'ing', 'er'],
    'teach': ['er', 'ing', 'es']
}

# Morphological generation: create words from roots and suffixes.
print("Morphological Generation:")

generated_words = []

for root, suffixes in morphology.items():
    for suffix in suffixes:
        # Apply a few basic English spelling rules.
        if root.endswith('e') and suffix == 'ing':
            word = root[:-1] + suffix
        elif root.endswith('ch') and suffix == 's':
            word = root + 'es'
        else:
            word = root + suffix

        generated_words.append(word)
        print(root, "+", suffix, "=", word)

# Morphological analysis: find roots and suffixes in generated words.
print("\nMorphological Analysis:")

for word in generated_words:
    found = False

    for root, suffixes in morphology.items():
        for suffix in suffixes:
            # Reconstruct the expected form using the same rules.
            if root.endswith('e') and suffix == 'ing':
                expected = root[:-1] + suffix
            elif root.endswith('ch') and suffix == 's':
                expected = root + 'es'
            else:
                expected = root + suffix

            # Match the observed word with its root and suffix.
            if word == expected:
                print(word, "-> Root:", root, ", Suffix:", suffix)
                found = True
                break

        if found:
            break
```

Sample output

```
Morphological Generation:
play + s = plays
play + ed = played
play + ing = playing
play + er = player
walk + s = walks
walk + ed = walked
walk + ing = walking
walk + er = walker
teach + er = teacher
teach + ing = teaching
teach + es = teaches

Morphological Analysis:
plays -> Root: play , Suffix: s
played -> Root: play , Suffix: ed
playing -> Root: play , Suffix: ing
player -> Root: play , Suffix: er
...
```

Explanation:

• **Generation:** Root + suffix → word form.

• **Analysis:** Word form → root + suffix.

These are inverse processes in morphology. The program illustrates common suffixes and a few spelling rules; it is not a complete English morphological analyzer.